DIGITAL TWIN FACTORY – Real-Time Industrial Monitoring and Predictive Maintenance System

A Project Based Learning Report Submitted in partial fulfilment of the requirements for

the award of the degree

Bachelor of Technology

in the Department of Electronics and Communication Engineering

Full Stack Web Development (25SC2008E)

Submitted by

2510040011: Y. SWAJITH

2510040017: C. VANSHIKA

2510040018: Y. SOMANATH

2510040118: PRABHU CHARAN

Under the guidance of

Dr. Chiranjeevi

Koneru Lakshmaiah Education Foundation

Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCTOBER 2026

Abstract

Digital Twin Factory is a web-based industrial monitoring platform designed to represent factory machines digitally and display their telemetry, operating status, alarms, analytics and maintenance information in one dashboard. The system combines Internet of Things (IoT) telemetry, MQTT messaging, a Python ingestion service, PostgreSQL storage, a FastAPI backend and a React-based frontend with a three-dimensional factory view. An ESP32/Wokwi device is intended to represent Machine 01, while a simulator can generate virtual telemetry for Machines 02–04 during demonstration.

Incoming sensor messages contain machine identity and readings such as temperature, humidity, vibration and pressure. The ingestion service validates messages, stores readings and applies alarm rules. The API exposes snapshots, historical readings, alarms, threshold configuration and maintenance work orders. The dashboard receives near-real-time updates through Server-Sent Events (SSE), allowing operators to inspect machine health and react to abnormal conditions.

The project demonstrates how a digital representation of factory assets can improve visibility, support earlier investigation of abnormal readings and organize maintenance actions. Its prediction and health-index features use statistical heuristics and should be treated as decision support rather than a safety-certified failure predictor. Production OEE is not claimed because verified PLC production counters are not part of the current implementation.

Keywords: Digital Twin, Industrial IoT, MQTT, PostgreSQL, FastAPI, React, Predictive Maintenance, SSE.

List of Figures

Figure 1. Overall system architecture

Figure 2. Telemetry processing and alarm workflow

Figure 3. Dashboard and digital-twin demonstration flow

Figure 4. Predictive-maintenance and work-order workflow

List of Tables

Table 1. Main technology stack

Table 2. Main API endpoints and responsibilities

Table 3. Demonstration scenarios and expected observations

Table 4. Limitations and future enhancements

Table of Contents

Abstract

1. Introduction

2. Methodology

3. Experiments and Demonstration

4. Results and Discussion

5. Conclusion and Future Work

References

Appendix A – Repository and Setup

DIGITAL TWIN FACTORY – Real-Time Industrial Monitoring and Predictive Maintenance System

1. INTRODUCTION

Manufacturing environments contain multiple machines whose condition changes continuously. Temperature, vibration, pressure and other sensor values can indicate changes in operating conditions. When such information is scattered across separate devices or reviewed only after a problem occurs, operators may have difficulty identifying abnormal behaviour quickly. A digital-twin dashboard brings telemetry and machine status into a shared view, helping users understand the current state of assets and inspect historical patterns.

The Digital Twin Factory project provides a software prototype for monitoring multiple factory machines. Each machine is represented as a digital asset with sensor readings, connectivity status, alarm state and health information. The system uses a message-based IoT pipeline so that data collection, processing, storage, API access and presentation are separated into manageable services.

1.1 Problem Statement

A factory monitoring workflow needs a consolidated way to receive machine telemetry, identify threshold breaches, preserve event history and present the information to an operator. The project addresses this need with a browser dashboard connected to an IoT data pipeline. It is a prototype and does not replace certified industrial control, emergency-stop or safety systems.

1.2 Objectives

Collect and validate telemetry from an ESP32/Wokwi device and virtual machine simulators.

Transport machine messages through MQTT using a consistent topic and payload contract.

Store readings, alarms, thresholds and maintenance work orders in PostgreSQL.

Expose current state and historical information through a FastAPI REST API.

Push updates to the dashboard using Server-Sent Events (SSE).

Display a 3D factory twin, fleet health, alarms, analytics, history and maintenance workflows.

Demonstrate statistical trend/anomaly indicators while documenting their limitations.

1.3 Scope

The current prototype covers software-based monitoring and demonstration workflows. It includes sensor values for temperature, humidity, vibration and pressure; machine connectivity; configurable per-machine thresholds; alarm acknowledgement and history; health scoring; trend-based maintenance hints; and stored work orders. It does not claim integration with a live production PLC, validated failure-prediction model or real production-count OEE.

1.4 Practical Usefulness

A single screen can help a maintenance or operations team see which machines are reporting, review abnormal readings, acknowledge alarms and track follow-up work. A trend estimate may prompt an operator to inspect a bearing or other component before the reading reaches a configured limit. The value of the prototype is improved visibility and a structured workflow; any real-world deployment would require site-specific validation, security controls and integration testing.

2. METHODOLOGY

2.1 System Architecture

The data path is: edge device or simulator → MQTT broker → Python MQTT subscriber → PostgreSQL → FastAPI → React dashboard. The backend subscriber validates incoming payloads, stores sensor readings and runs alarm logic. The API serves REST endpoints and a Server-Sent Events stream. The frontend combines current data with machine cards, analytics, alarms and a 3D factory representation.

2.2 Technology Stack

2.3 Telemetry Contract

The documented MQTT topic pattern is digitaltwin/factory/machineNN, with a corresponding machineNN/status topic for retained online/offline status. A typical telemetry message contains machine_id, temperature, humidity, vibration and pressure. The timestamp may be included by the publisher; the database records arrival time. A payload whose machine identifier does not match the topic is rejected.

Example JSON: {"machine_id": 1, "temperature": 27.4, "humidity": 55.1, "vibration": 2.7, "pressure": 1013.2}

2.4 Alarm and Health Logic

The repository describes per-machine sensor thresholds, debouncing to reduce transient alerts, hysteresis for clearing alarms, acknowledgement metadata and alarm history. A computed health index combines distance from limits with trend/anomaly indicators. These are statistical heuristics, not a trained machine-learning model. Thresholds must be configured and validated for each real asset.

2.5 Predictive Maintenance and Work Orders

Trend estimation and z-score anomaly detection are used to surface a maintenance hint, such as a sensor approaching its warning threshold. A user can create a maintenance work order from a recommendation and track it through open, in-progress and completed states. The prediction is an indication for human review, not a guarantee that a machine will fail at a particular time.

2.6 Live Dashboard Updates

The API exposes a Server-Sent Events endpoint at /api/stream. The browser can receive server-pushed snapshots while REST endpoints support data retrieval and actions. SSE is the implementation documented for this version; this report does not describe the system as using WebSockets.

2.7 Security and Deployment Considerations

The project supports environment-based configuration for database credentials, MQTT credentials, CORS origins and an optional API key. A shared API key placed in a frontend build can be inspected by users and is not a suitable secret for a real plant. A production deployment should place privileged actions behind a server-side authentication layer or reverse proxy/SSO, use TLS, protect broker credentials and restrict device topic permissions. The repository also includes Docker Compose files, although its README notes that the Docker build was not tested in the original authoring environment.

3. EXPERIMENTS AND DEMONSTRATION

3.1 Demonstration Setup

Configure PostgreSQL and apply the supplied database schema.

Install backend Python dependencies and configure the database password in the environment.

Start the FastAPI API once and start the MQTT subscriber once.

Install frontend dependencies and run the Vite development server.

Start the simulator for virtual machines; use Wokwi for M01 if available. Do not simulate a machine that is already publishing from a real/Wokwi device.

Open the dashboard and confirm that machine status and readings are updating before presenting.

3.2 Experiment 1 – Live Sensor Monitoring

Aim: demonstrate the flow of telemetry from an edge/simulated source to the dashboard. Procedure: start the simulator, open the overview and machine pages, and observe sensor values and connectivity state. Expected observation: the dashboard displays incoming readings and machine status after the complete backend/database pipeline is available.

3.3 Experiment 2 – Alarm Generation and Acknowledgement

Aim: show how a threshold breach becomes an operator-visible alarm. Procedure: use the demo simulator's fault behaviour or a safe test threshold, open Alerts, inspect the alarm details and acknowledge an alarm. Expected observation: an alarm appears after the configured rule conditions are met and the acknowledgement is recorded. Avoid claiming an alarm has been tested unless it is observed live.

3.4 Experiment 3 – Trend and Predictive-Maintenance Hint

Aim: explain how trend/anomaly indicators can help prioritize inspection. Procedure: open Analytics or Maintenance and inspect the displayed trend estimate and recommendation. Expected observation: the interface may estimate when a sensor is approaching a configured limit. This is a statistical demonstration, not a validated prediction of actual equipment failure.

3.5 Experiment 4 – Work-Order Lifecycle

Aim: demonstrate converting a maintenance recommendation into tracked work. Procedure: create a work order, move it through the available statuses and verify the persisted record. Expected observation: maintenance actions can be followed from creation to completion in the dashboard.

3.6 Experiment 5 – History and Analytics

Aim: show historical readings and fleet-level analytics. Procedure: select a machine and sensor, review the history chart and compare current fleet indicators. Expected observation: historical telemetry supports review of trends rather than relying only on the latest reading.

3.7 Suggested Review-4 Demo Sequence

Introduce the factory-monitoring problem and the intended users.

Show the architecture and point to the telemetry source, MQTT broker, ingestion service, database, API and dashboard.

Open Overview and Machines to show current readings and connectivity.

Trigger or wait for a demo alarm, then acknowledge it.

Open Analytics/Maintenance to explain trends and work orders.

Finish with the 3D factory view, practical benefits and honest limitations.

4. RESULTS AND DISCUSSION

The project architecture and source code provide a coherent end-to-end prototype for machine telemetry monitoring: MQTT messaging, backend ingestion, PostgreSQL persistence, REST access, SSE updates and a React dashboard. The documented feature set includes machine-level thresholds, alarm debouncing/hysteresis, alarm acknowledgement/history, machine health indicators, trend-based maintenance hints, historical charts and work-order tracking.

The primary outcome is operational visibility: a user can inspect a fleet of digital machine representations and navigate from an abnormal reading to an alarm and a maintenance action. A successful live demonstration depends on correctly configured database credentials, database schema, MQTT connectivity and running services. A health endpoint that reports database unavailable, missing tables or authentication errors indicates a deployment/configuration issue rather than a successful end-to-end run.

4.1 Demonstration Scenarios

4.2 Limitations

The health index and trend predictions are statistical heuristics; they are not a trained or safety-certified failure model.

Production OEE is not implemented because verified production counts and ideal cycle data from a PLC are not available. Do not present simulated OEE as measured plant performance.

The dashboard's 'Availability (24 h)' refers to data availability, not production uptime.

A frontend environment variable included in a browser bundle cannot protect an API key as a secret.

The README states that the Docker build itself was not run in the original authoring environment; verify deployment behaviour before claiming a tested container release.

The simulator and public MQTT broker are for demonstration; a real deployment should use a controlled broker, authentication, TLS and appropriate access controls.

5. CONCLUSION AND FUTURE WORK

Digital Twin Factory demonstrates an integrated approach to industrial IoT monitoring by connecting telemetry sources, MQTT messaging, data ingestion, relational storage, a web API and an interactive dashboard. The prototype makes machine state, alarms, analytics and maintenance actions easier to inspect in a single interface. It is suitable as a PBL demonstration of full-stack development and IoT integration, provided the implementation is described accurately and the live setup is verified before review.

5.1 Future Work

Integrate verified PLC counters and cycle-time data before implementing production OEE.

Replace heuristic prediction with a model trained and validated on representative machine-failure data.

Add server-side identity, role-based access, TLS, secret management and audited authorization for write actions.

Deploy with a managed PostgreSQL instance, tested container builds, health checks and backup/restore procedures.

Validate sensor calibration, thresholds and alarm timing against real equipment and maintenance expertise.

Expand test coverage for data ingestion, reconnect behaviour, alarm rules and work-order workflows.

REFERENCES

[1] Digital Twin Factory v4 project repository and README. https://github.com/swajith-26/Digital-twin-Factory

[2] FastAPI documentation. https://fastapi.tiangolo.com/

[3] PostgreSQL documentation. https://www.postgresql.org/docs/

[4] MQTT specifications and resources, OASIS. https://mqtt.org/

[5] React documentation. https://react.dev/

[6] Vite documentation. https://vite.dev/

[7] Three.js documentation. https://threejs.org/docs/

[8] Recharts documentation. https://recharts.org/

APPENDIX A – REPOSITORY AND SETUP

Repository: https://github.com/swajith-26/Digital-twin-Factory

The README provides Windows PowerShell setup instructions, environment-variable details, MQTT topic format, API endpoint summaries and testing commands. Follow the repository README as the source of truth for current commands and configuration.

Before final submission, confirm that the repository includes the final Review-4 PPT, this report, setup instructions and all source files. Remove local secrets, database passwords and private .env files from the repository.

Layer	Role in the project
Edge / simulation	ESP32 on Wokwi for M01; simulator can generate virtual telemetry for M02–M04.
Messaging	MQTT topics carry JSON sensor readings and retained online/offline status.
Ingestion and rules	Python subscriber validates payloads, persists readings and applies threshold/alarm logic.
Database	PostgreSQL stores machine data, sensor readings, alarms, thresholds and work orders.
API and live updates	FastAPI provides REST endpoints and an SSE stream.
User interface	React dashboard provides overview, machine views, 3D twin, analytics, alerts, history and maintenance.

Technology	Purpose
Python	Telemetry ingestion, alarm processing and backend services.
MQTT	Lightweight publish/subscribe messaging for IoT telemetry.
PostgreSQL	Persistent relational storage for readings and operational records.
FastAPI	REST endpoints, health checks and SSE streaming.
React + Vite	Frontend application and dashboard components.
Three.js / React Three Fiber	Three-dimensional factory and machine visualization.
Recharts / Lucide React	Charts, indicators and interface icons.

Technology	Purpose
Wokwi / ESP32 simulator	Demonstration source for device telemetry.

Scenario	What to inspect	Evidence to capture during final run
Telemetry	Overview / machine detail	Screenshot showing changing readings and online status.
Alarm	Alerts page	Screenshot of a real demo alarm and its state.
Analytics	Charts / trend indicator	Screenshot of history or a displayed maintenance hint.
Work order	Maintenance page	Screenshot of work-order status transition.
3D twin	Factory view	Screenshot of the rendered digital factory.

Embedded figure